

CHAPTER 02

Communication & Core Concepts

How microservices find, talk to, and work with each other

Inter-service Communication

API Gateway

Service Discovery

Data Isolation

What We'll Cover

01

Communication Overview

Synchronous vs asynchronous — the two ways services talk

02

The API Gateway

The single entry point — first stop for every request

03

Synchronous Communication

HTTP/REST — services calling each other directly

04

Asynchronous Communication

Message queues — fire and forget, no waiting

05

Service Discovery

How services find each other at runtime

06

Data Isolation

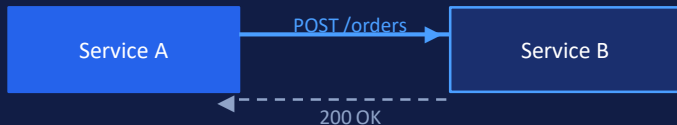
Why each service must own its own database

Two Ways Services Talk

⚡ Synchronous

HTTP / REST

Service A calls Service B and waits for the response before continuing.

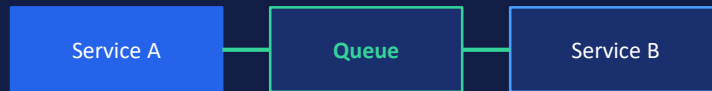


- ✓ Simple to implement
- ✓ Easy to debug & trace
- ✗ Caller is blocked while waiting
- ✗ Tight runtime dependency

✉ Asynchronous

Message Queue (RabbitMQ, Azure Service Bus...)

Service A publishes an event and moves on. Service B picks it up whenever it's ready.



- ✓ Caller doesn't wait (non-blocking)
- ✓ Services are fully decoupled
- ✗ Harder to debug & trace
- ✗ More infrastructure needed

The Single Front Door

Without a gateway, clients need to know the address of every service. The gateway is the only address they need.



 **Routing**  **Authentication**  **Rate Limiting**

Popular tools: YARP (C# / .NET) · Ocelot (C# / .NET) · Kong · NGINX · AWS API Gateway · Azure API Management

HTTP Calls Between Services

From our C# store example — OrderService calling ProductService to check stock:

```
OrderService.cs

// Step 1 – register the HTTP client
builder.Services.AddHttpClient("ProductService",
    c => c.BaseAddress = new Uri("http://localhost:5001"));

// Step 2 – call ProductService
var client = httpFactory.CreateClient("ProductService");
var product = await client.GetFromJsonAsync<Product>(
    $"/products/{req.ProductId}");
```

What's happening:

1

Register the client

Tell ASP.NET where ProductService lives

2

Inject the factory

IHttpClientFactory creates a configured client

3

Make the HTTP call

GET /products/{id} → deserialize JSON response

4

Handle the result

If null → product doesn't exist → return 400

⚠ What if ProductService is down? → OrderService fails too. This is the risk of synchronous calls.

Events & Message Queues

Scenario: order is placed → notification should be sent. Does OrderService really need to wait for the email?



Event: "OrderPlaced"

```
{
  "orderId": 42,
  "customerEmail": "student@example.com",
  "productName": "Laptop",
  "total": 2599.98
}
```

Why this is better here:

- ✓ OrderService finishes instantly — doesn't wait
- ✓ NotificationService can be down, event is queued
- ✓ Add more consumers (SMS, push) without changing OrderService

How Services Find Each Other

✗ The Problem

In our demo we hardcoded:

```
http://localhost:5001
```

What if the service moves to a different server or port?

What if you run 5 instances for load balancing?

What if it's in Kubernetes with a dynamic IP?

✓ Service Discovery

A service registry keeps a live map of who is running and where.

Service Registry

ProductService → 10.0.0.4:5001

OrderService → 10.0.0.7:5002

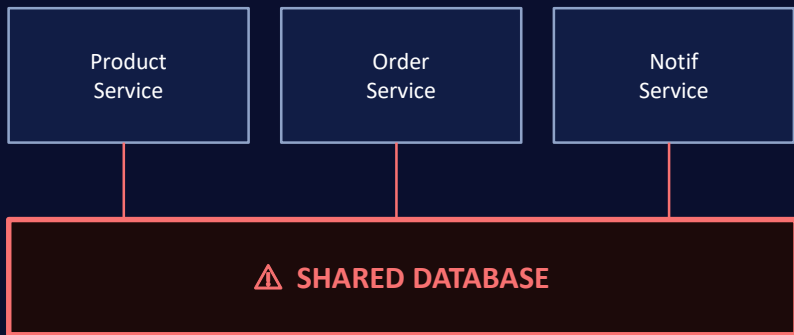
NotifService → 10.0.0.9:5003

Popular tools: Consul · Kubernetes DNS · Eureka (Java) · AWS Cloud Map

Each Service Owns Its Data

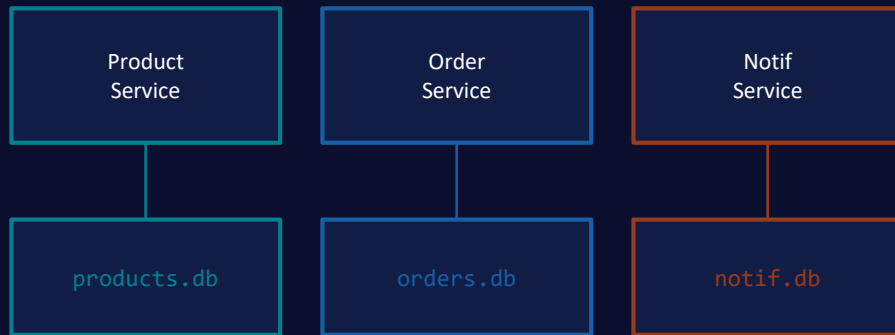
Rule: no two services may share a database or call each other's database directly.

✗ Wrong — shared DB



- ✗ Schema change breaks all services
- ✗ One slow query affects everyone
- ✗ Can't scale services independently

✓ Correct — database per service



- ✓ Each service can use a different DB type
- ✓ Schema changes stay isolated
- ✓ Scale storage independently per service

Key Takeaways

01 Sync vs Async

Two ways services talk: HTTP (simple, waits for response) and message queues (non-blocking, fully decoupled).

02 API Gateway

One entry point for all clients. Handles routing, authentication, and rate limiting before requests reach services.

03 Synchronous (HTTP)

Services call each other directly and wait. Simple to debug but creates runtime dependencies.

04 Asynchronous (Events)

Services publish events to a broker. Consumers react at their own pace — fully decoupled.

05 Service Discovery

A service registry keeps a live map of addresses. No more hardcoded ports in production.